

Teoría de la Información y Codificación

Práctica 1: Creación código libre de prefijos con árboles binarios y cálculo de Kraft-Millan

José A. Montenegro Montes

5 de febrero de 2026

1. Enunciado

La práctica se centra en la construcción de códigos libre de prefijo utilizando un árbol, en este caso binario. El usuario pasará los parámetros del código deseado y obtendremos una codificación si es posible.

El alumno deberá realizar el método que realiza el cálculo de Kraft-McMillan (KM) descrito en las transparencias del tema 2 (pagina 24). Verifique los resultados con el Ejercicio 7 (pagina 29) de las transparencias.

El número KM nos permite verificar si es posible crear un código libre de prefijos según unos parámetros datos.

El objetivo principal que persigue la práctica es observar que mediante el calculo de Kraft-McMillan sabemos de antemano si se puede o no construir el código con los parámetros seleccionados, sin tener que calcularlos. Observaremos el tiempo que tarda realizar el calculo con el árbol y el tiempo en calcular con Kraft-McMillan, tal y como muestra la figura 1.

Tratando el tema de optimización observaremos que mediante una pequeña modificación en el cálculo de Kraft-McMillan (utilizando una alternativa a Math.pow) podemos bajar notablemente el tiempo empleado en su cálculo (figura 2).

2. Conclusiones

El objetivo de esta práctica es desarrollar código libres de prefijo mediante recorrido en un árbol binario, estructura que le resultará familiar al alumno.

Observaremos la utilidad de aplicar la fórmula de Kraft-McMillan para hacer más eficiente nuestro algoritmo.

Finalmente, veremos que aunque en la mayoría de los casos, es preferible utilizar la librería estándar de Java, en ocasiones puntuales, una implementación propietaria puede mejorar los tiempos .

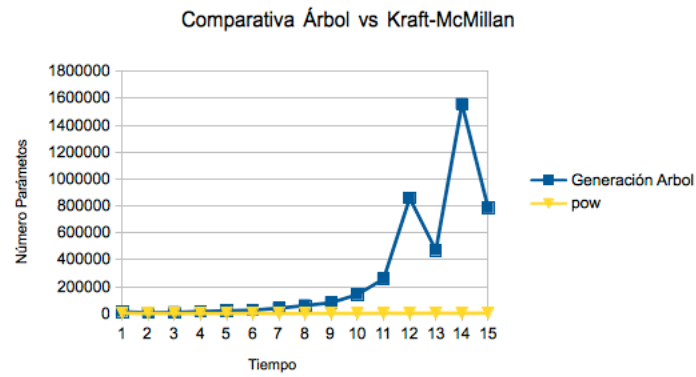


Figura 1: Generación código Árbol vs Kraft-McMillan

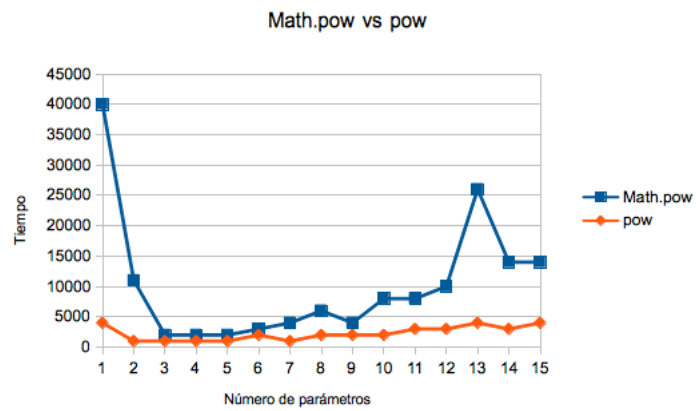


Figura 2: Utilización Math.pow vs pow en Kraft-McMillan

3. Código

Clase BinaryTreeCod

```

/*****
 * Practica 1 Teoría de la Información y Codificación. (Acentos eliminados)
 *
 * La practica muestra la construcción de códigos libre de prefijo utilizando un
 * árbol, en este caso binario.
 * La construcción es realizada basado en los parámetros por nivel.
 *
 * Tal y como detallamos en las transparencias del tema 2 (pagina 24) realizamos
 * el cálculo de Kraft-McMillan.
 * Verifique los resultados con el Ejercicio 7 (pagina 29) de las transparencias.
 *
 * El objetivo principal que persigue la practica es observar que mediante el
 * cálculo de Kraft-McMillan sabemos de antemano si se puede o no construir el
 * código con los parámetros seleccionados.
 *
 * Observaremos el tiempo que tarda realizar el cálculo con el árbol y el tiempo
 * en calcular con Kraft-McMillan.
 *
 * Además observaremos que realizando una modificación en el cálculo de
 * Kraft-McMillan, utilizando una
 * alternativa a Math.pow.
 *
 * @author joseamontenegromontes
 *
 * Version 1: 4 Octubre 2013
 *
 */
package Practical1;

import java.util.ArrayList;
import java.util.List;

public class BinaryTreeCod {

    Node root=new Node("");
    int levelTree=1;

    /***/

```

```

* Clase Privada Node.
*
* Almacena los datos de los nodos del arbol.
* @author joseamontenegromontes
*
*/

    private class Node {

        String name;
        Boolean used=false;

        Node leftChild=null;
        Node rightChild=null;

/*****
* Constructor
* @param name
*/
        Node(String name) {

            this.name = name;

        }

/*****
* Imprime los valores del nodo. Para depuracion.
*/
        public String toString() {
            if (name.isEmpty()) return "raiz";
            else return name + " = "+used;
        }

/*****
* Establece el nodo de la derecha como utilizado
*/
        public boolean setRUsed (){

            return setUsed (rightChild);

        }

/*****
* Establece el nodo de la izquierda como utilizado
*/
        public boolean setLUsed (){

            return setUsed (leftChild);

```

```

    }

    /**
     * Auxiliar para setLUsed y setRUsed
     */
    private boolean setUsed (Node node){
        boolean returnValue=true;

        if (node.isUsed()) returnValue=false;
        else node.used=true;

        return returnValue;
    }

    /**
     * Conocemos si un nodo esta utilizado
     */
    public boolean isUsed (){
        return used;
    }

}

    /** Fin clase privada Node */

    /**
     * Constructor
     * @param levelp niveles del arbol
     */
    public BinaryTreeCod (int levelp){
        levelTree=levelp;
        generate(root,levelTree);
    }

    /**
     * Auxiliar para crear el arbol por niveles
     * @param focusNode
     * @param level
     */
    private void generate(Node focusNode,int level) {

```

```

        if (level != 0) {

            String izq=focusNode.name.concat("0");
            String dch=focusNode.name.concat("1");

            focusNode.leftChild = new Node(izq);
            focusNode.rightChild = new Node(dch);

            generate(focusNode.leftChild,level-1);
            generate(focusNode.rightChild,level-1);

        }

    }

    /**
     * Construímos un código libre de prefijos según los parámetros
     * @param n
     * @throws TreeException
     */

    public boolean buildCodeParameters(int []n) throws TreeException {
        int size= n.length;
        boolean result=true;

        if (size > levelTree ) throw new TreeException ("Not enough levels!!");

        for (int i=0;i<size;i++){
            if(n[i]>0) result=levelFreePrefix(i+1, n[i]); //Eliminate unnecessary
            if (!result) return false;
        }
        return true;
    }

    /**
     * Auxiliar de buildCodeParameters. Rellena los n elementos de un nivel.
     * @param level
     * @param n
     */

    private boolean levelFreePrefix(int level, int n) {
        int used=levelAuxPrefixFree(root,level, n);

        if (used==0) return true;
        else return false;
    }

```

```
}
```

```
/******
```

```
 *      AUXILIAR
```

```
 *
```

```
 * Auxiliar de levelFreePrefix. Rellena usedLevel elementos de un nivel dado level, desde n
```

```
 * @param focusNode
```

```
 */
```

```
private int levelAuxPrefixFree(Node focusNode,int level, int usedLevel) {
```

```
    if (level==1){
```

```
        if (usedLevel>0 & !focusNode.isUsed())           if (focusNode.setLUsed()) usedLevel--; //l
```

```
        if (usedLevel>0 & !focusNode.isUsed())           if (focusNode.setRUsed())
```

```
            usedLevel--;
```

```
    }
```

```
    else{
```

```
        if (usedLevel>0 & !focusNode.leftChild.isUsed() ){
```

```
            usedLevel=levelAuxPrefixFree(focusNode.leftChild,level-1, usedLevel);
```

```
        }
```

```
        if (usedLevel>0 & !focusNode.rightChild.isUsed() ) {
```

```
            usedLevel=levelAuxPrefixFree(focusNode.rightChild,level-1, usedLevel);
```

```
        }
```

```
    }
```

```
    return usedLevel;
```

```
}
```

```
/******
```

```
 * Imprime el arbol generado
```

```
 */
```

```
    public void printBinaryTree(){
```

```
        printBinaryTree(root, 0);
```

```
    }
```

```
/******
```

```
 *      Auxiliar de printBinaryTree
```

```
 *
```

```
 * @param root
```

```
 * @param level
```

```
 */
```

```

private static void printBinaryTree(Node root, int level){
    String output=null;

    if(root==null) return;

    printBinaryTree(root.leftChild, level+1);

    if(level!=0){
        for(int i=0;i<level-1;i++)
            System.out.print("\t");

        if (root.isUsed()) output=root.name;
        else output="x";
        System.out.println("|-----"+output);
    }
    else
        if (root.isUsed()) System.out.println(root.name);
        else
            System.out.println("x");

    printBinaryTree(root.rightChild, level+1);
}

/*****
 * Verifica si el arbol es libre de prefijo
 */
public boolean prefixFree(){

    return prefixFreeAux(root,false);

}

/*****
 * Auxiliar de prefixFree
 * @param node
 * @param checked
 * @return
 */
public boolean prefixFreeAux(Node node,boolean checked){
    boolean check=false;
    boolean end=false;

    if (node==null) return true;

```



```

        if (!node.name.isEmpty()) { //Eliminate root node.
            check=node.isUsed();

            if (check && checked) {
                return false; //Non prefix Free.
            }
            else {
                checked=check|checked; // Super or working node selected
            }
        }

        end=prefixFreeAux(node.leftChild,checked);
        if (end) end=prefixFreeAux(node.rightChild,checked);

        return end;
    }

    
    /*****
    *          Calculo de la potencia binaria. 2^x
    *
    * @param x
    * @return
    */
    */
    public double pow (int x){

        return (1 << x);
    }

    
    /*****
    * Calculo de kraftMcMillan segun parametros del codigo usando Math.pow
    * @param ni
    * @return
    */
    */
    public double kraftMcMillan(int [] ni) {

        double K = 0;
        //ToDo: Tu codigo aqui
        return K;
    }

    
    /*****
    * Calculo de kraftMcMillan segun parametros del codigo usando nuestro pow
    * @param ni
    */
    */

```

```

    * @return
    */
    public double kraftMcMillanOptimization(int [] ni) {
        double K = 0;
        //ToDo: Tu codigo aqui
        return K;
    }

    /*****
    * Imprime los parametros generados.
    * ToDo: parameters puede ser eliminados para no evitar confusiones. Deben ser los mismos q
    * para construir el arbol. Se pueden almacenar como atributo de la clase.
    * @param parameters
    */

    public void printCodes(int [] parameters) {
        List <String> CodeList=getCodes();
        int size=CodeList.size();
        int sizep=parameters.length;

        System.out.print("\nCode obtained with ");

        for (int i = 0; i < sizep; i++) {
            System.out.print(parameters[i]+" ");
        }

        System.out.println(" parameters: ");

        for (int i = 0; i < size; i++) {
            System.out.print(" "+CodeList.get(i)+" ");
        }
    }

    /*****
    * Obtiene los codigos generados.
    * @param ni
    * @return
    */
    public List <String> getCodes() {
        List <String> CodeList=new ArrayList<String>();

        CodeList= inOrderTraverseTree(root,CodeList);

        return CodeList;
    }

```

```

/*****
 * Auxiliar getCodes. Recorrido en orden, almacena solamente los nodos seleccionados.
 * @param ni
 * @return
 */
private List <String> inOrderTraverseTree(Node focusNode,List <String> code) {

    if (focusNode != null) {
        if (focusNode.isUsed()) code.add(focusNode.name);
        code=inOrderTraverseTree(focusNode.leftChild,code);
        code=inOrderTraverseTree(focusNode.rightChild,code);
        return code;
    }

    return code;
}

/*****
 * Funcion para verificar la practica.
 * @param parameters. Crear codigo segun unos parametros.
 * @param printTree. Decido si quiero imprimir o no el arbol.
 */
static public void Test (int[]parameters,boolean printTree){

    boolean result=true;
    double KMvalue=0;
    long ini,fin; //Time variables.

    int leng= parameters.length;
    BinaryTreeCod code = new BinaryTreeCod(leng);

    try {
        ini=System.nanoTime();
        result=code.buildCodeParameters(parameters);
        fin=System.nanoTime();

        if(result){
            System.out.println("BinaryTreeMethod:Found prefix free code in "+(fin-ini)+ " nanos");

            if (printTree) code.printBinaryTree(); //Print tree
        }
    }
}

```

```

        ini=System.nanoTime();
        boolean isPrefixFree=code.prefixFree();
        fin=System.nanoTime();

        System.out.println("Verifying prefix free: "+isPrefixFree+" in "+(fin-ini)+ " nanosec

    }
    else
        System.out.println("BinaryTreeMethod: I can not generate prefix free code usign this

} catch (TreeException e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
}

        ini=System.nanoTime();
        KMvalue = code.kraftMcMillan(parameters);
        fin=System.nanoTime();

        System.out.println("kraft- McMillan Regular      Value: "+KMvalue+" in "+(fin-ini)+

        ini=System.nanoTime();
        KMvalue = code.kraftMcMillanOptimization(parameters);
        fin=System.nanoTime();

        System.out.println("kraft- McMillan Optimization Value: "+KMvalue+" in "+(fin-ini)+

        code.printCodes(parameters);

        System.out.println("");
    }
    /*****
    *
    * @param args
    */

    public static void main(String[] args) {

        boolean printTree=true;

        int [] parametersC1={0,1,4,3};
        BinaryTreeCod.Test(parametersC1,printTree);
    }
}

```

Clase TreeException Java.

```
package Practical1;

public class TreeException extends Exception {

    public TreeException(String message){
        super(message);
    }

}
```
